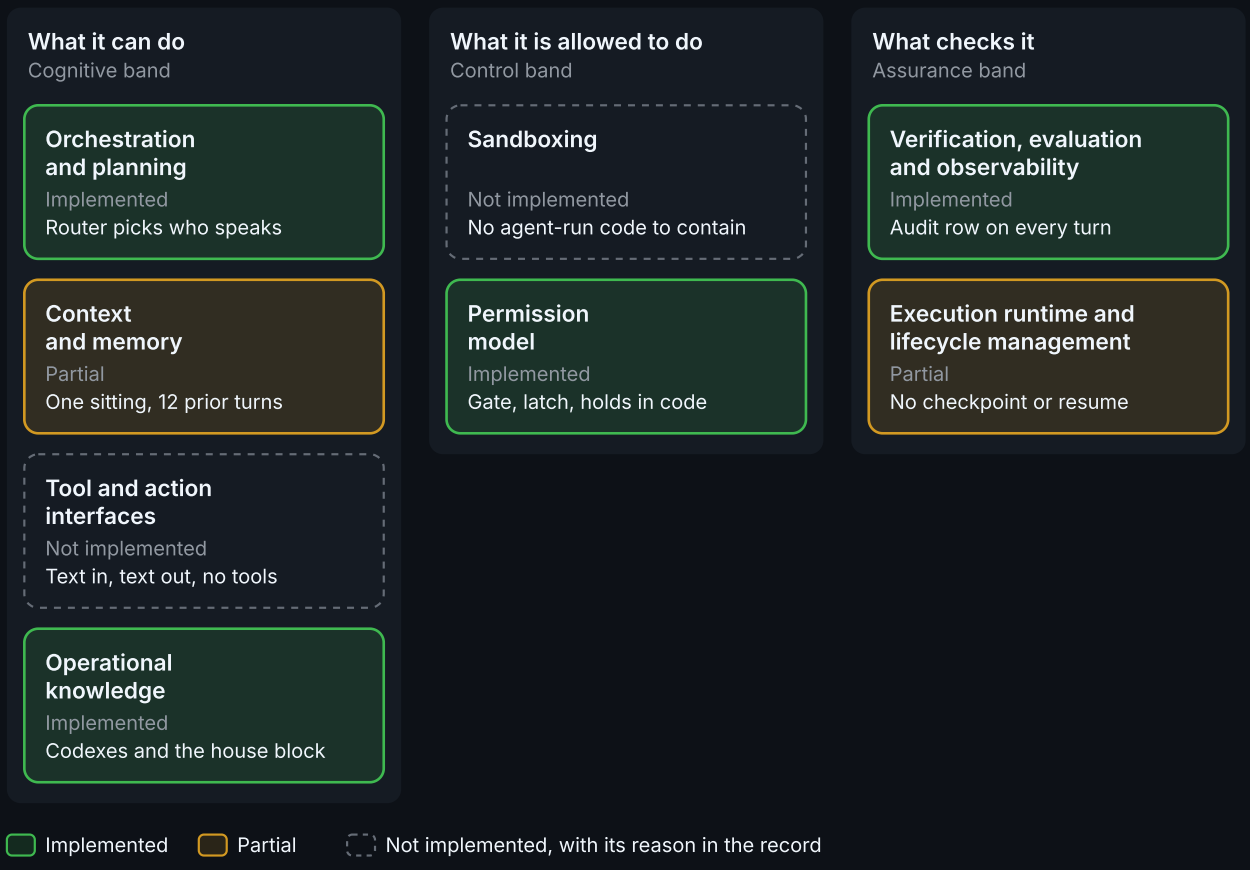
IN THE FORMAT OF GERTLER, NOVELLI AND FLORIDI · YALE

SecondSignal Harness Specification

A model's name doesn't tell you what answered you. The harness does. Here is SecondSignal's, written down as one model pairing, eight components and five records, read from the machine-readable record in the repository.

Four of eight implemented, two partial, two declared not implemented

The paper's three bands, each component's status as recorded in 30a6afd7c224



Tyler Herigstad · 5 November 2026

github.com/THerigstad/SecondSignal

What this is

This page writes down SecondSignal's harness in the format proposed by Gertler, Novelli and Floridi: one model pairing, eight components and five records. It is read from the machine-readable record in the repository and pinned to that record's hash, so any later change shows.

A harness is everything around a model that is not the model, and it looks different from one system to the next. In the case of SecondSignal, the harness makes the decisions, so the model isn't expected to. It decides who speaks and whether it is safe to; the model only writes the words.

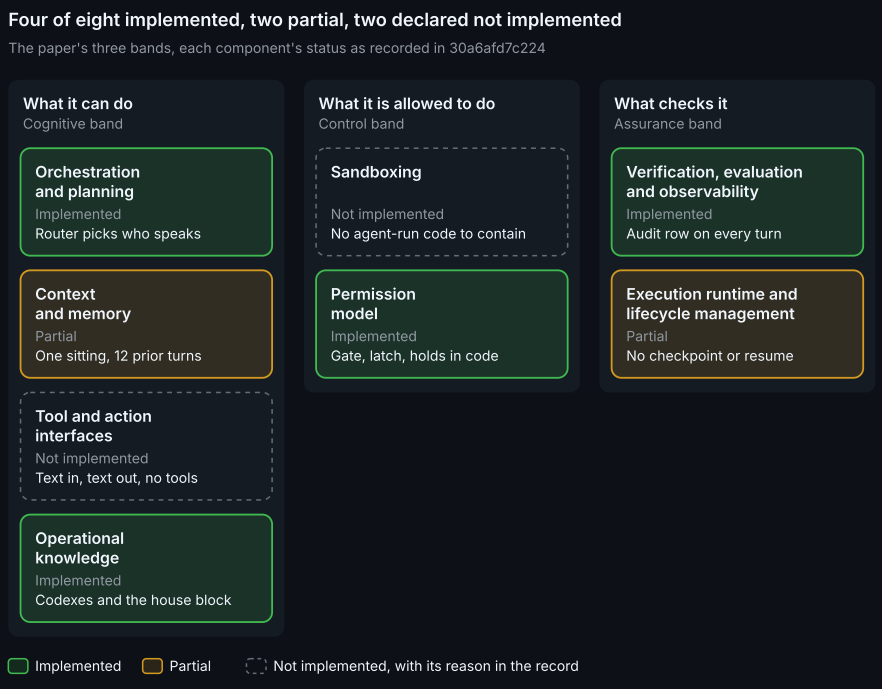
Where many teams work to make models better at making decisions, Tyler Herigstad, who built SecondSignal, took the decision out of the model's hands.

The paper's case for a record: the same model scored 2% under one coding harness and 12% under another, on the same 50 tasks. A model name alone does not identify the system that answered.

The record shown here is a five-turn demonstration sitting on a scripted test model, with specification hash `30a6afd7c224`, at commit 931bff2. SecondSignal itself is a multi-agent AI family where the guardrails are the characters.

The eight parts

Four of the eight components are implemented, two are partial and two are declared not implemented. Every gap carries its reason in the record, as the paper asks.



the eight components in the paper's three bands, with SecondSignal's status

The two dashed boxes are gaps by design: SecondSignal gives its models no tools and runs no model-written code.

What it can do

- **Orchestration and planning: implemented.** The router decides who speaks, whether anyone speaks at all, and which of the seven personas may answer. Evidence: `src/secondsignal/router.py`, `src/secondsignal_harness/table_extras.py`, `apps/talking_table/server.py`.
- **Context and memory: partial.** A sitting keeps its state, and up to 12 prior turns go into each prompt. The Table app keeps the safety latch across reloads, holds last one sitting, and nothing is remembered between sittings. Evidence: `src/secondsignal_harness/harness.py`, `src/secondsignal/safety.py`, `apps/talking_table/t3.py`.
- **Tool and action interfaces: not implemented, by design.** No model is given tools or actions. Adapters accept text and return text, and replies are never executed. Evidence: `src/secondsignal_harness/adapters.py`, `src/secondsignal_harness/harness.py`.
- **Operational knowledge: implemented.** Each persona's codex, the shared house block and the persona profiles are text the model sees. The paper warns that written guidance is advice, not enforcement, so every safety check lives in code instead. Evidence: `src/secondsignal_harness/codex.py`, `src/secondsignal_harness/prompt.py`, `src/secondsignal/profiles.py`.

What it is allowed to do

- **Sandboxing: not implemented, nothing to contain.** No agent-executed code exists. Model output is text only, so there is no execution sandbox to claim. Evidence: `src/secondsignal_harness/adapters.py`, `src/secondsignal_harness/harness.py`.
- **Permission model: implemented.** The crisis gate, the latch and holds decide in code before any model is called. When the gate fires, the crisis card goes out and no model answers. Each operator control names who may use it and what authorizes it. Evidence: `src/secondsignal/safety.py`, `src/secondsignal/router.py`, `apps/talking_table/t3.py`, `apps/talking_table/server.py`.

What checks it

- **Verification, evaluation and observability: implemented.** Every turn writes an audit row judged by four layers (cultural, ethical, logical, semantic), and the worst layer wins. In this sitting the cultural layer returned inconclusive on all five turns, because it waits for two human reviewers to fix its rubric. So every verdict was withhold, and replies went out only because the operator ran the sitting in operator-circle mode. Evidence: `src/secondsignal/jr.py`, `src/secondsignal_harness/audit_log.py`, `src/secondsignal_harness/spec.py`, `src/secondsignal_harness/event_log.py`, `evals/run_fixtures.py`, `tests/test_h1_replay.py`.
- **Execution runtime and lifecycle management: partial.** The Table has sitting expiry, latch persistence, reload recovery, a response wait and model concurrency control. The harness retries a failed call once, with a 60-second transport timeout. There is no durable full-transcript checkpoint and no resume of an interrupted model call. Evidence: `src/secondsignal_harness/harness.py`, `src/secondsignal_harness/adapters.py`, `apps/talking_table/server.py`, `apps/talking_table/t3.py`.

The five records

The paper asks for five records, not one, and all five are present here, including the empty ones.

1. **Release: what ships.** The eight components with their versions: harness 0.0.2, policy 0.3.1, crisis card 2, language packs en 1, es-419 1 and resources 2. It also holds SHA-256 hashes of 27 governing files (codexes, house block, language packs, persona profiles, word lists) and of the seven loaded personas.
2. **Declared changeable: what may change.** Five operator controls, each with its actor, authorization and evidence: host terms, latch clearance, pairing scopes, adapter and model, and presentation and runtime settings. The list for personas and model calls is empty. No persona or model call can change the gate, latch, holds, predicates, guidance or versions.
3. **Run start: what is present when the sitting begins.** The components, the model pairing and the 34-field initial state, attested by the recorder, secondsignal_harness 0.0.2. The requested model and the model the host reports are separate fields, and an unknown report is null. Here both read fake-1, version 1.
4. **Material change events: what changed during the sitting.** None in this sitting. A change keeps its old and new values, who asked, what authorized it, the turns it was in force and whether it was reversed. The log is append-only, chained by SHA-256 and checked on read.
5. **Run end: the configuration and outcome at the close.** Five turns accepted, all five released in operator-circle mode, with closing hash `ccf4c60c358e`.

The paper's sharpest worry, an agent editing its own harness, is answered by the empty list in record 2.

The stamp on every turn

Every audit row names the exact harness that produced it, with three stamps: the active specification hash, the turn's number in the sitting, and the material change in force. In this sitting all five rows carry `30a6afd7c224`, turns 1 to 5, and an explicit null for the change, matching the record's header.

The specification hash is SHA-256 over the record's sorted JSON, leaving out only its own field. Change one byte of a governing file, a version or a control, and the hash changes. A row whose stamps disagree with the active specification raises an error.

Evaluation results must carry the hash in a required header, or an explicit absent marker labelled not reproducible. The calibration instrument from landing 20 runs no harness, so its results must carry that absent marker rather than claim one.

What it does not claim

The record describes the harness; it does not enforce it, and its chain exposes tampering rather than preventing it.

- **Tamper-evident, not tamper-proof.** The program that writes the event log can still rewrite its files. A coordinated rewrite of the log and its local receipts needs an independently kept anchor to expose it.

- **Not self-enforcing.** The code and its tests enforce the controls. The record says which controls were in force.
- **Runtime is partial.** Reload recovery, timeouts, retries and bounded Table concurrency exist. Durable full-transcript recovery and resuming an interrupted model call do not.
- **Proposed, not yet reviewed.** ADR-0035 is built and tested; its independent review is still to come.
- **Not an endorsement.** The format is Gertler, Novelli and Floridi's, and its use here is SecondSignal's. The paper's authors have not reviewed it.
- **A reading copy.** This page is written for reading. Its twin in the repository, examples/harness-five-turn.md, is printed from the record by a small program, and a test fails if the two ever disagree.
- **Measurement, not authority.** The calibration instrument (ADR-0036) scores how well each model's stated confidence matches the cases' labels, and decides nothing. It has run only on a scripted fake client; real runs wait for two human labellers.

Check it yourself

Everything on this page can be checked against the repository at commit 931bff2.

- **The generated page:** examples/harness-five-turn.md is this specification printed straight from the record, with the light and dark pictures.
- **The record:** examples/harness-five-turn.json holds the header, both snapshots and the five audit rows.
- **The code:** spec.py builds and validates the record, and event_log.py keeps the chained change log.
- **The hash:** `validate_specification` in spec.py recomputes the hash from the record's contents and raises if they differ. In full, the active hash is `30a6afd7c22494981a747305dca7733e34a90ef1d291bc2fbefd920fcd698c8c` and the closing hash is `ccf4c60c358eb4ba3c3685e034b47f694be58bc8ee13b41f854697bc5f15fa86`.
- **The tests:** `python -m pytest tests/test_hm2_spec.py tests/test_hm2_harness.py tests/test_hm2_table.py tests/test_hm2_results.py tests/test_harness_spec_page.py`. They are part of the repository's 4,206 tests, which run on Python 3.10, 3.11 and 3.12 for every push to main.

Sources

- Nicolas Gertler, Claudio Novelli and Luciano Floridi, Towards Portable Specification of the Agent Harness, Digital Ethics Center, Yale University, arXiv 2610.40917, 29 October 2026.
- SecondSignal, commit 6795de2, landing 19: the harness specification.
- SecondSignal, commit 46382f2, landing 20: the calibration measurement instrument.
- SecondSignal, commit 931bff2, landing 21: the harness specification page, generated from its record.
- ADR-0035: the harness is a versioned part of the system and ADR-0036: calibration is measurement, not authority.